

Session 2 Homework — Recently Viewed Opportunities

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you must submit.

Context

This extends the same B4F Hub project. It is **not** the separate weekly mini-project (handed out the same day) and **not** a separate project — it continues from where Session 2 ended.

Right now, when you open an opportunity's detail page, nothing remembers that you were there. If you go back to the Opportunities list and want to return to something you looked at a minute ago, you have to search or scroll for it again.

Goal

Remember which opportunities a visitor recently looked at, and let them jump back with one click.

What to build

A `RecentlyViewedContext` (the same Context + Provider + hook pattern you built twice in class) that records opportunity views and displays them as clickable links.

CORE (required)

1. A new Context, `RecentlyViewedContext`, whose Provider owns a list of `{ id, title }` entries and a `recordView(entry)` function.
2. Wrap the app in the new Provider (same place as the other two Providers from class).
3. Call `recordView` from the opportunity detail page, exactly once a matching opportunity is found — never for a bad id.
4. A new component, `RecentlyViewedList`, that reads the list from Context and renders it as a "Recently viewed" strip of clickable links (using `Link`, not `<a href>`) — or renders nothing at all when the list is empty.
5. Show `RecentlyViewedList` on the Opportunities panel.
6. **CORE does not need to prevent duplicates, reorder anything, or limit the list's length** — that is STRETCH's job. Recording the same opportunity twice may add it twice, or you may choose to skip an id you've already recorded — either is acceptable for CORE.

STRETCH (required)

1. Change `recordView` so that:
 - viewing an opportunity that is **already** in the list moves it to the **front** instead of adding a duplicate;

- the list never grows past **5** entries — the oldest entry is dropped once a 6th distinct opportunity is viewed.

CHALLENGE (required)

1. Add a `clearAll` function to the Context, and a **Clear** button on `RecentlyViewedList` that calls it.
2. Also render `RecentlyViewedList` on the opportunity detail page itself — but it must **exclude** the opportunity currently being viewed (an opportunity should never appear in its own "recently viewed" list).

Rules

Allowed — everything from Session 1 and Session 2: `createContext`, a Provider component, `useContext` (directly or through your own custom hook), `Link`, `useParams`, and ordinary array methods (`.filter`, `.some`, `.slice`, `spread`).

Not yet — Redux/Redux Toolkit, `useReducer`, any Node/Express or server change, and any browser storage (the list does not need to survive a page refresh — that is intentionally out of scope).

Test it

- Visit `/opportunities/101`, then `/opportunities/205`, then go to `/opportunities`: a "Recently viewed" strip shows both, with working links back to each detail page.
- Visit 6 different opportunities in a row: the strip never shows more than 5, and the most recently viewed one is always first.
- Re-visit an opportunity already in the list: it moves to the front instead of appearing twice.
- Open an opportunity's detail page: the "recently viewed" strip there lists everything EXCEPT that same opportunity.
- Click **Clear**: the list empties everywhere it is shown.
- Visit a made-up id such as `/opportunities/99999`: nothing is added to the list.

Done when

- ☐ CORE: views are recorded and shown as clickable links on the Opportunities panel.
- ☐ STRETCH: the list is capped at 5 and reorders most-recent-first.
- ☐ CHALLENGE: a Clear button works, and the detail page shows the list excluding itself.
- ☐ `npm run build` and `npm run lint` (inside `client/`) both pass.
- ☐ No console errors in the browser.

Working time

About 45–60 minutes total across all three tiers (roughly 20 CORE / 15 STRETCH / 20 CHALLENGE).